X_405082
Advanced Computer Networks

Fundamentals

Lin Wang (lin.wang@vu.nl)

Period 2, Fall 2020

VU VRIJE UNIVERSITEIT AMSTERDAM

Part of the content is adapted from Laurent Vanbever and Ankit Singla

Course outline

Warm-up

- **Fundamentals** 👈
- Forwarding and routing
- Network transport

Data centers

- Data center networking
- Data center transport

Programmability

- Software defined networking
- Programmable forwarding

Video

- Video streaming
- Video stream analytics

Networking and ML

- Networking for ML
- ML for networking

Mobile computing

- Wireless and mobile

Learning objectives

What do we want from the network and how to achieve them? (**principles**)

What are the basic **data structures** for networking?

What do we want from the network?

Performance: latency? bandwidth?

Reliability, availability, security?

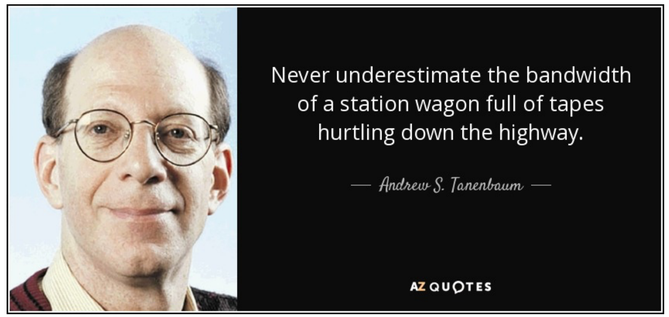
Flexibility, manageability?

Others?

Network performance

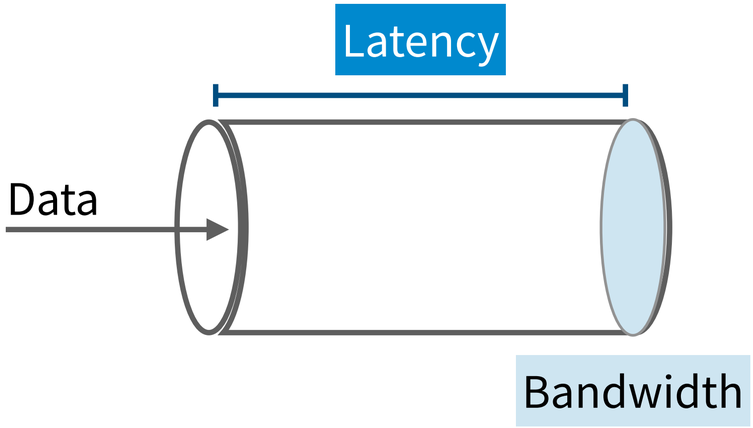
Bandwidth

- Definition: maximum rate of data transfer across a given network path
- "*Never underestimate the bandwidth of a station wagon full of tapes hurtling down the highway.*" [1]



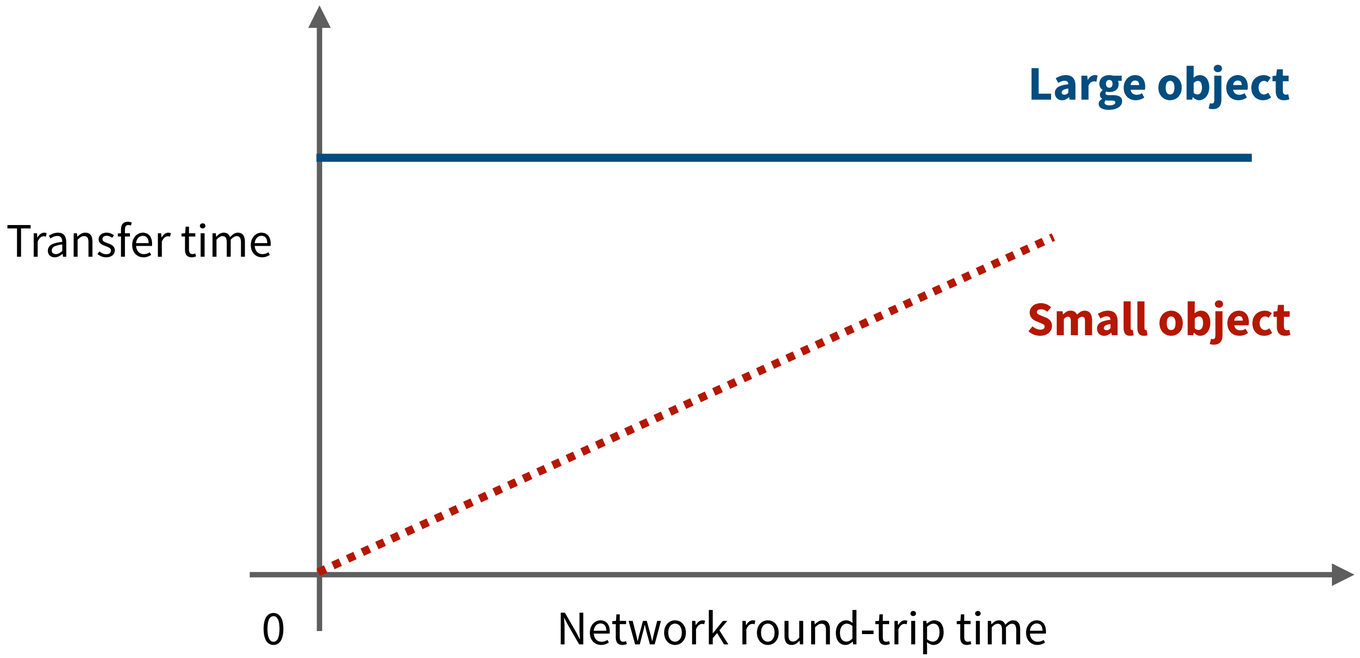
Latency

- Definition: the amount of time it takes to deliver some data from the source to the destination across the network
- "Latency lags bandwidth" — David A. Patterson



[1] Andrew S. Tanenbaum paraphrasing Dr. Warren Jackson, Director, University of Toronto Computing Services (UTCS) circa 1985

Bandwidth vs. latency



Assume a network link with 10Gbps bandwidth and 1ms latency?
How long does it take to transfer 1B, 100KB, or 10GB of data?

With **TCP**: 100KB = 1460B * (0 + 2 + 4 + 8 + 16 + 32 + 6) → 7ms in the best case, 0.0143Gbps

Do you know how we come to this?

Performance metric

Flow completion time

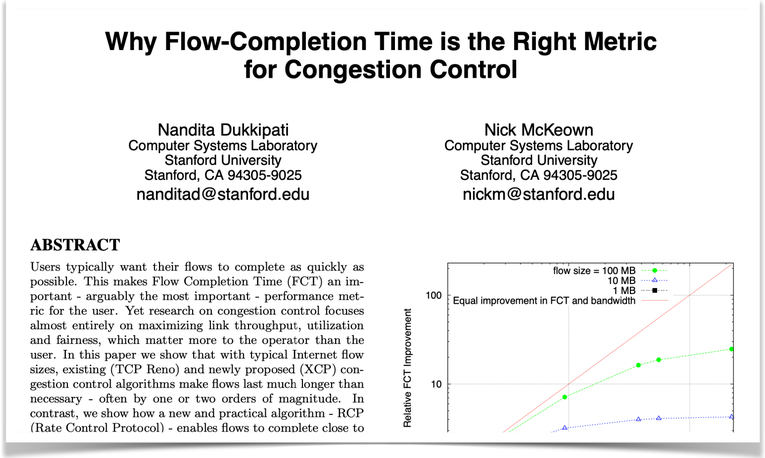
- How long does it take to complete a traffic flow?
- How long does it take to complete a set of correlated flows (co-flows)?

What else?

- How long does https://www.google.com/?q=cool+network take?
- What is the best video quality I can watch, without the annoying "buffering"?
- How to guarantee the per-frame latency (e.g., 20ms) in AR?

Not just about performance

- But also **consistent**, **predictable** performance



Why Flow-Completion Time is the Right Metric for Congestion Control

Nandita Dukkipati
Computer Systems Laboratory
Stanford University
Stanford, CA 94305-9025
nanditad@stanford.edu

Nick McKeown
Computer Systems Laboratory
Stanford University
Stanford, CA 94305-9025
nickm@stanford.edu

ABSTRACT

Users typically want their flows to complete as quickly as possible. This makes Flow Completion Time (FCT) an important - arguably the most important - performance metric for the user. Yet research on congestion control focuses almost entirely on maximizing link throughput, utilization and fairness, which matter more to the operator than the user. In this paper we show that with typical Internet flow sizes, existing (TCP Reno) and newly proposed (XCP) congestion control algorithms make flows last much longer than necessary - often by one or two orders of magnitude. In contrast, we show how a new and practical algorithm - RCP (Rate Control Protocol) - enables flows to complete close to

ACM SIGCOMM CCR 2006

What about fairness?

Suppose a network is flow fair. How useful is that?

Flow Rate Fairness: Dismantling a Religion

Bob Briscoe
BT Research & UCL
bob.briscoe@bt.com

ACM SIGCOMM CCR 2007

"Both the thing being allocated (rate) and what it is allocated among (flows) are **completely daft** — both unrealistic and impractical."

Food for thought:

- How to translate microbenchmarks to app-level metrics?
- How to make service providers accountable?
- How to improve the performance and approach fairness?

Network reliability

Three important considerations in network reliability

The end-to-end argument	The fate-sharing principle	Packet vs. circuit switching

The end-to-end argument

TCP provides reliable transport

What if no reliable transport is provided?

- Every application that needs reliability has to engineer it from scratch: programming burden, bugs…

What if the network layer tried to provide reliable delivery?

Reliable (or **unreliable**) transport

built on…

Best effort global packet delivery

Reliable ~~(or unreliable)~~ transport

built on…

Reliable global packet delivery

What are the problems?

The end-to-end argument

Problem #1: there are applications that require speedy delivery, even if it is lossy.

Problem #2: can the network even achieve reliable global packet delivery?

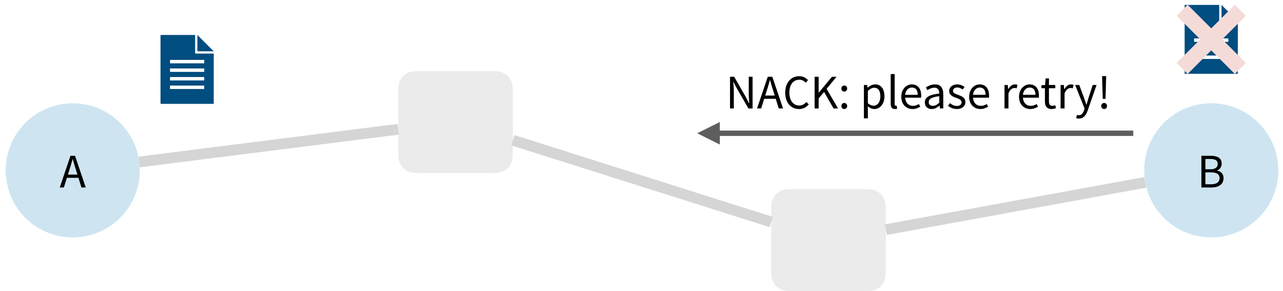
Check reliability at every step (on the network layer).

Problems: bugs, failures are a truth of life

The end-to-end argument

"If a function can only be correctly implemented end-to-end, it must be implemented in the end systems. Implementing it in the network can, at best, only be an optimization."

Examples?



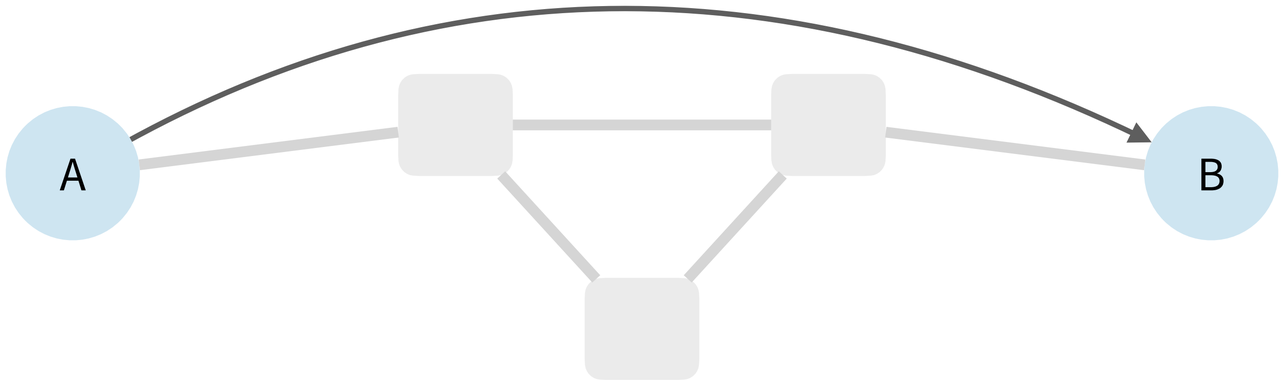
Allow unreliable steps (network layer is best-effort)
B checks correctness. On failure, B tell A to retry.

Can still fail, but only if A/B themselves fail → depends on what end-points themselves control

The fate-sharing principle

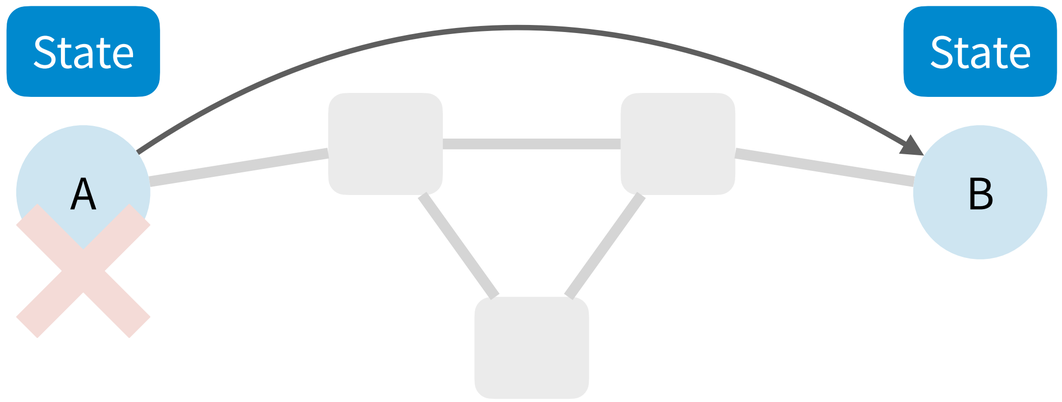
- To deal with potential failures, store critical system state at the nodes which rely on that state
- Only way to lose that state is if the node that relies on it fails, in which case it does not matter

Where to maintain A → B connection state?



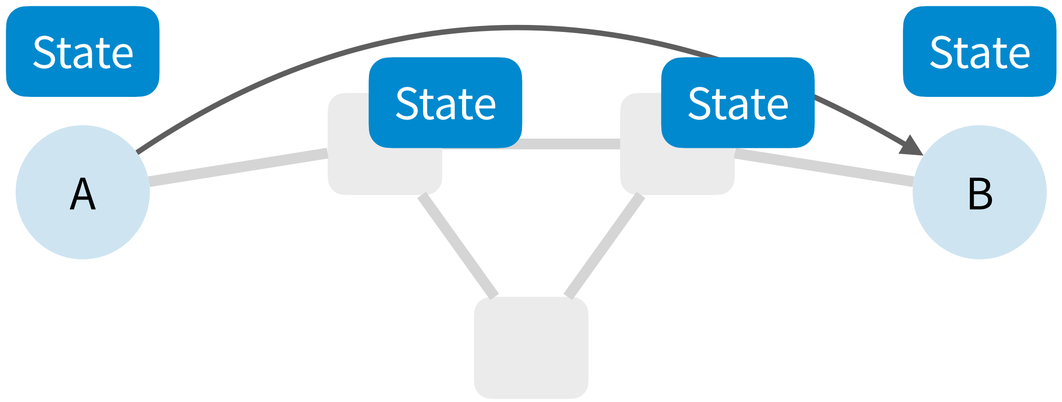
The fate-sharing principle

Keep state on end hosts



In the end-host fails, the connection state is irrelevant anyway

Keep state on network devices



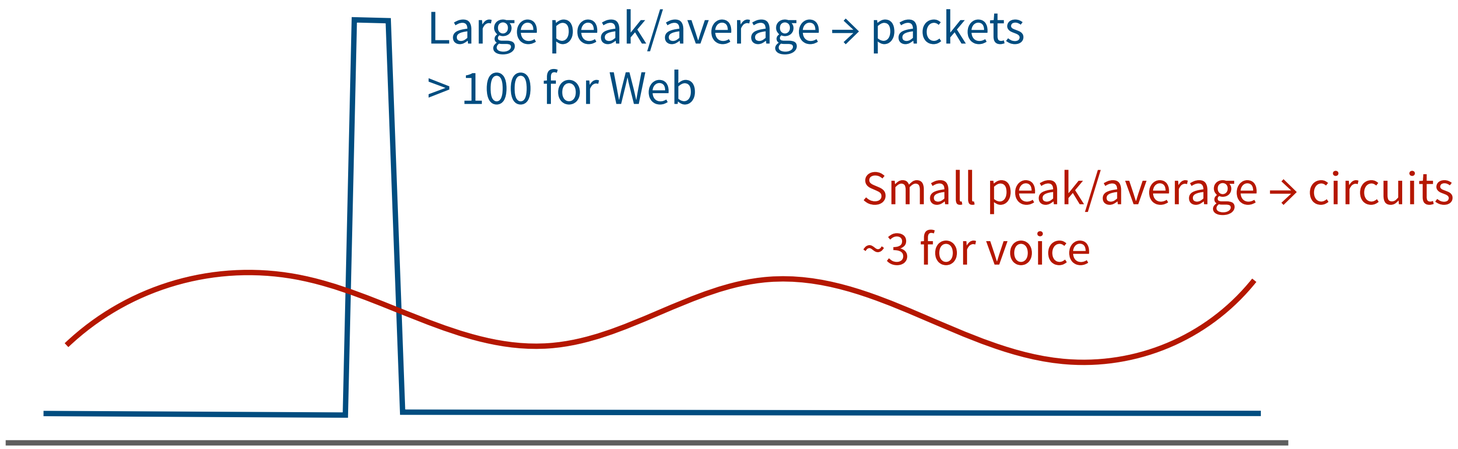
Failures of network devices require the state to be cleaned up or recreated → complex consistency issue!

Packet vs. circuit switching

Circuit switching	Packet switching
▪ Predictable performance	▪ Efficient use of resources ▪ Automatic, in-network rerouting on failures
▪ Wasteful, if packet is bursty and short ▪ Large latency for small messages, as they wait for circuits creation ▪ Require new circuit setup upon failure	▪ Unpredictable performance ▪ Requires buffer management and congestion control

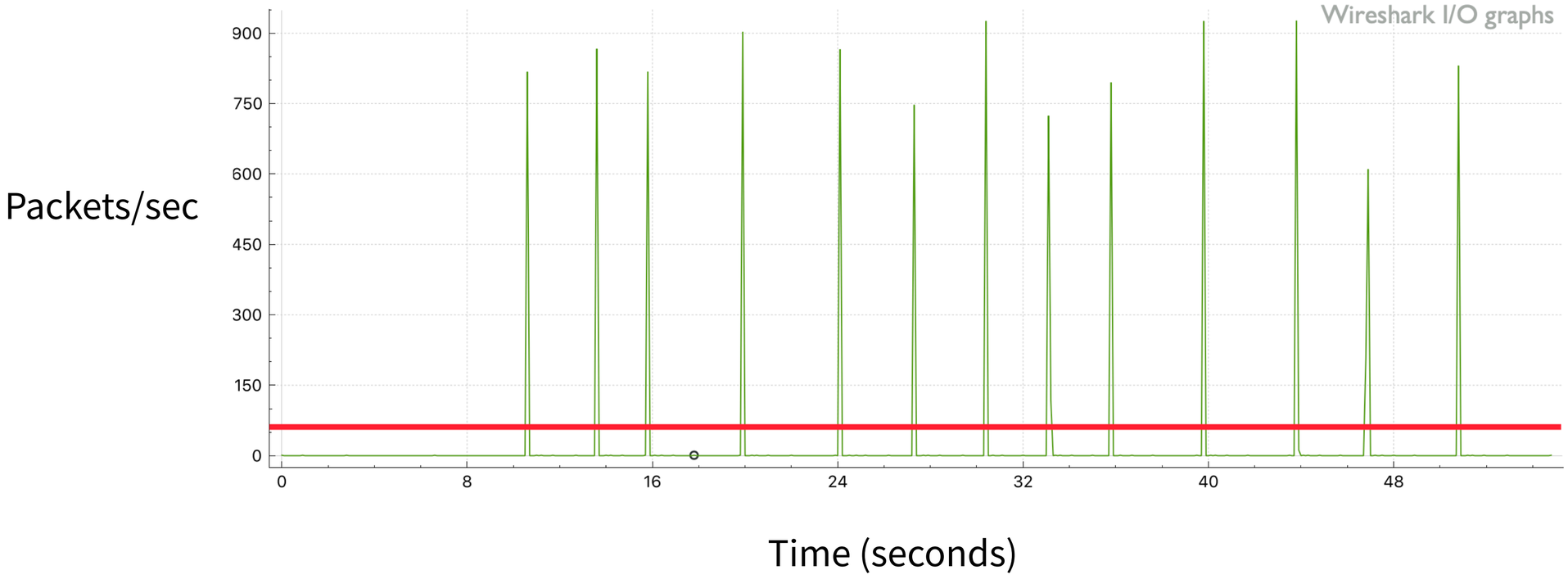
Packet switching beats circuit switching with respect to resilience and efficiency

Packet vs. circuit switching examples



Which pattern do you think modern video streaming (e.g., YouTube) will follow?

Video streaming today



Do you know why?

A perspective on networking

Why the Internet only just works

M Handley

The core Internet protocols have not changed significantly in more than a decade, in spite of exponential growth in the number of Internet users and the speed of the fastest links. The requirements placed on the net are also changing, as digital convergence finally occurs. Will the Internet cope gracefully with all this change, or are the cracks already beginning to show? In this paper I examine how the Internet has coped with past challenges resulting in attempts to change the architecture and core protocols of the Internet. Unfortunately, the recent history of failed architectural changes does not bode well. With this history in mind, I explore some of the challenges currently facing the Internet.

BT Technology Journal 2006

It was published 14 years ago. What have changed?

Questions?

Probabilistic data structures in networking

Matching and counting are popular operations in networking

- Matching on packet headers: membership, firewall, deep packet inspection
- Counting the number of packets that match certain criteria

Network devices are typically resource-constrained

- Memory is limited to few hundred KB of fast memory (e.g., TCAM)
- Computing power is also limited, so software implementation is slow in general

Probabilistic data structures are more suitable than deterministic ones, since they are generally **much faster** and **consumes less space**.

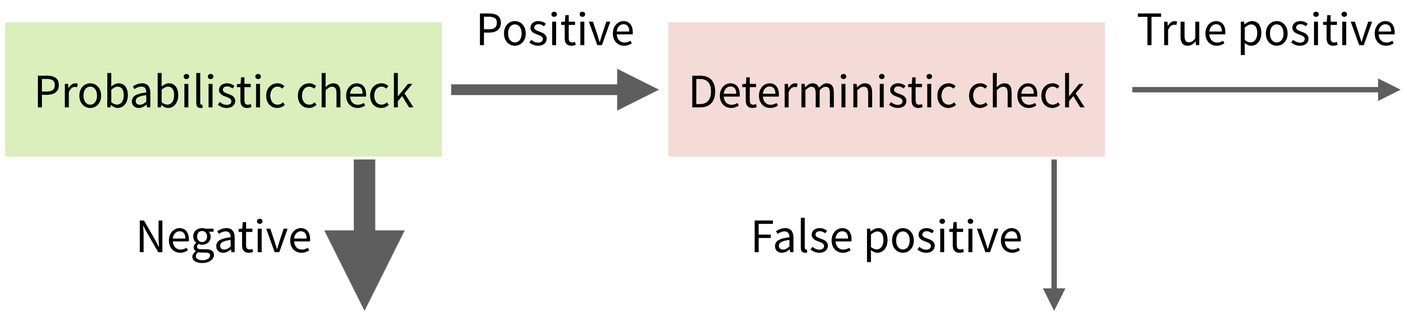
What are the problems with probabilistic data structures?

Problems with probabilistic data structures

False positives or false negatives

False-positives can be tolerated

- Often, they are not critically harmful, e.g., unnecessary packet transmissions and dropping
- In case it is critically important, we can first do the probabilistic check and perform deterministic check upon positives. With the upfront probabilistic check to filter out the negatives, the deterministic check would be much less frequent.



Find a needle in a haystack

Suppose we have an array in which we want to perform operations like “insert”, “find”, “delete”. What are the time complexities of these operations?

- Take “find” for example
- Unsorted array: O(n), sorted array: O(logn), binary search tree: O(logn)
- Can we do it in O(1)?

	Key	Value
0	Kimchi	Spicy cabbage
1	Kiwi	Austrialian fruit
2	Bread	Daily food

Find a needle in a haystack

Suppose we have an array in which we want to perform operations like “insert”, “find”, “delete”. What are the time complexities of these operations?

- Take “find” for example
- Unsorted array: O(n), sorted array: O(logn), binary search tree: O(logn)
- Can we do it in O(1)?

O(1) time: Array(“Kiwi”) = “Austrialian fruit”

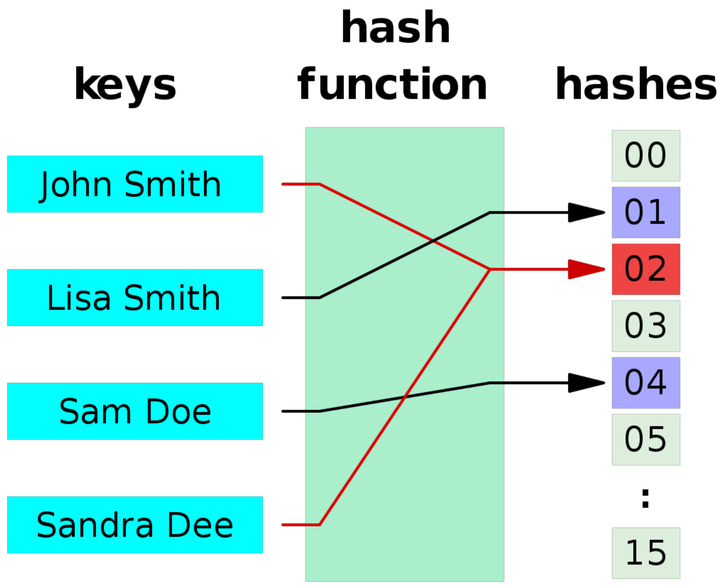
Need a **hash function** to covert the key to the index

	Key	Value
0	Kimchi	Spicy cabbage
1	Kiwi	Austrialian fruit
2	Bread	Daily food

Hash function

Properties of good hash functions

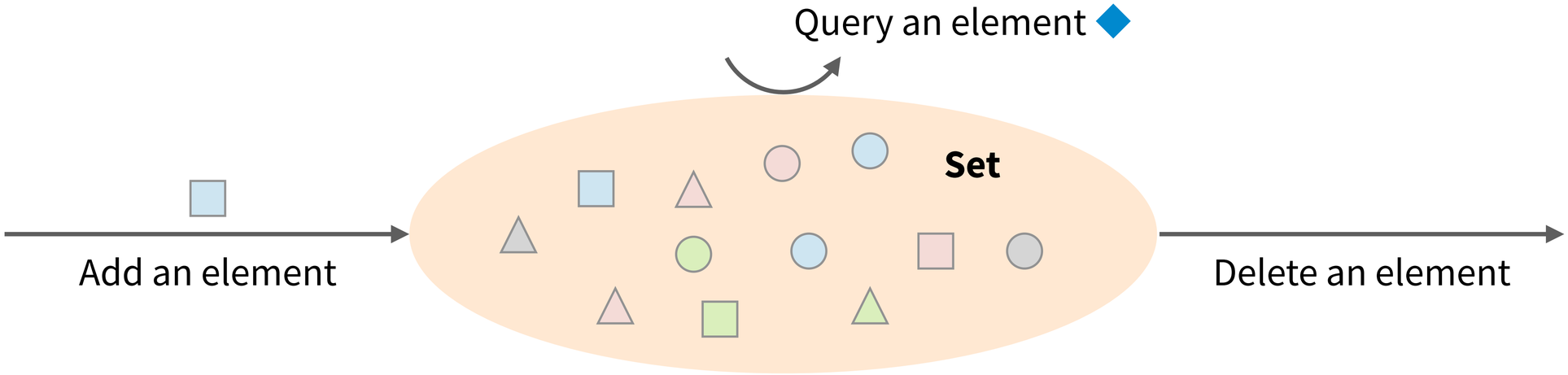
- Must return numbers 0,…, table_size
- Should be efficiently computable: O(1) time
- Should not waste space unnecessarily:
  - For every index, there is at least one key that hashes to it
  - Load factor lambda = (# of keys) / table_size
- Should minimize collisions (different keys hashing to the same index)



Examples hash functions

- Checksums, non-cryptographic hash functions (MurMur, FNV, Jenkins), cryptographic hash functions (MD5, SHA256, SHA512)

A recurrent problem in networking

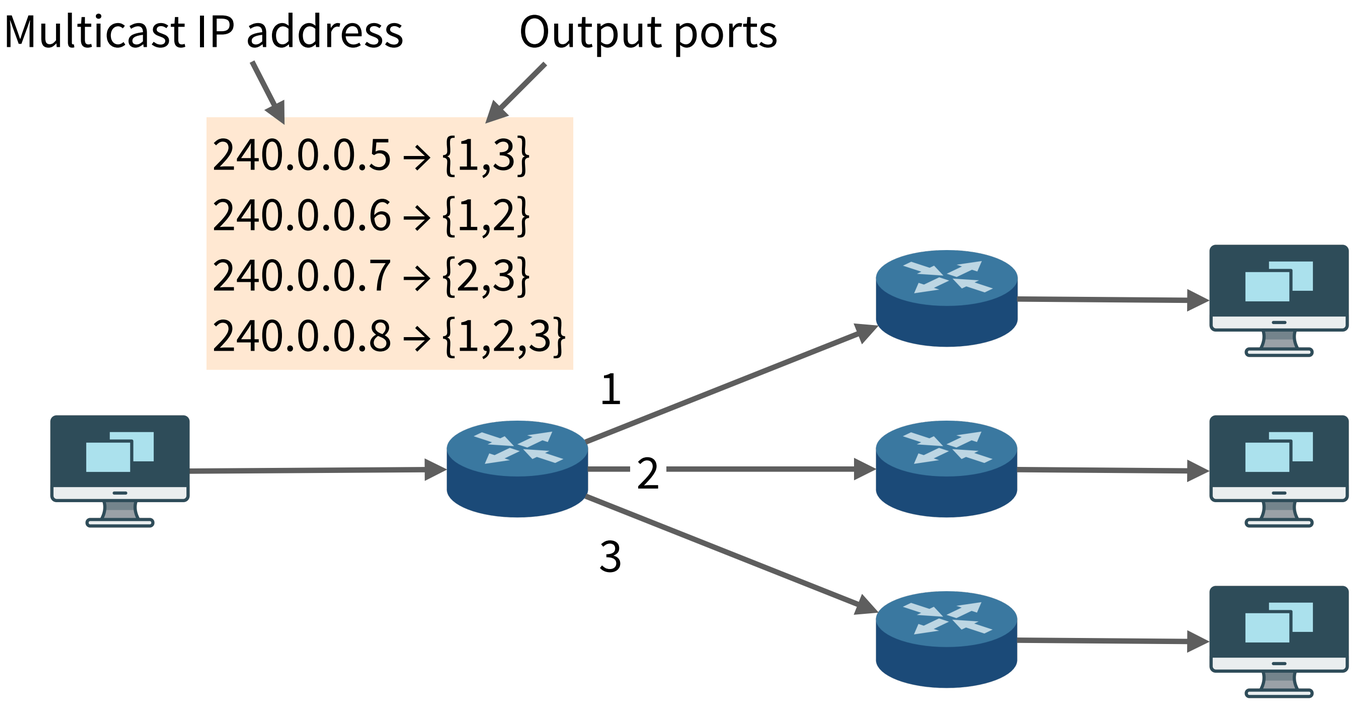


Traffic monitoring/classification,
membership determination

Common implementation strategies

	Strategy #0	**Strategy #1**	**Strategy #2**
Output	Deterministic	Deterministic	**Probabilistic**
Number of required operations	Deterministic	**Probabilistic**	Deterministic

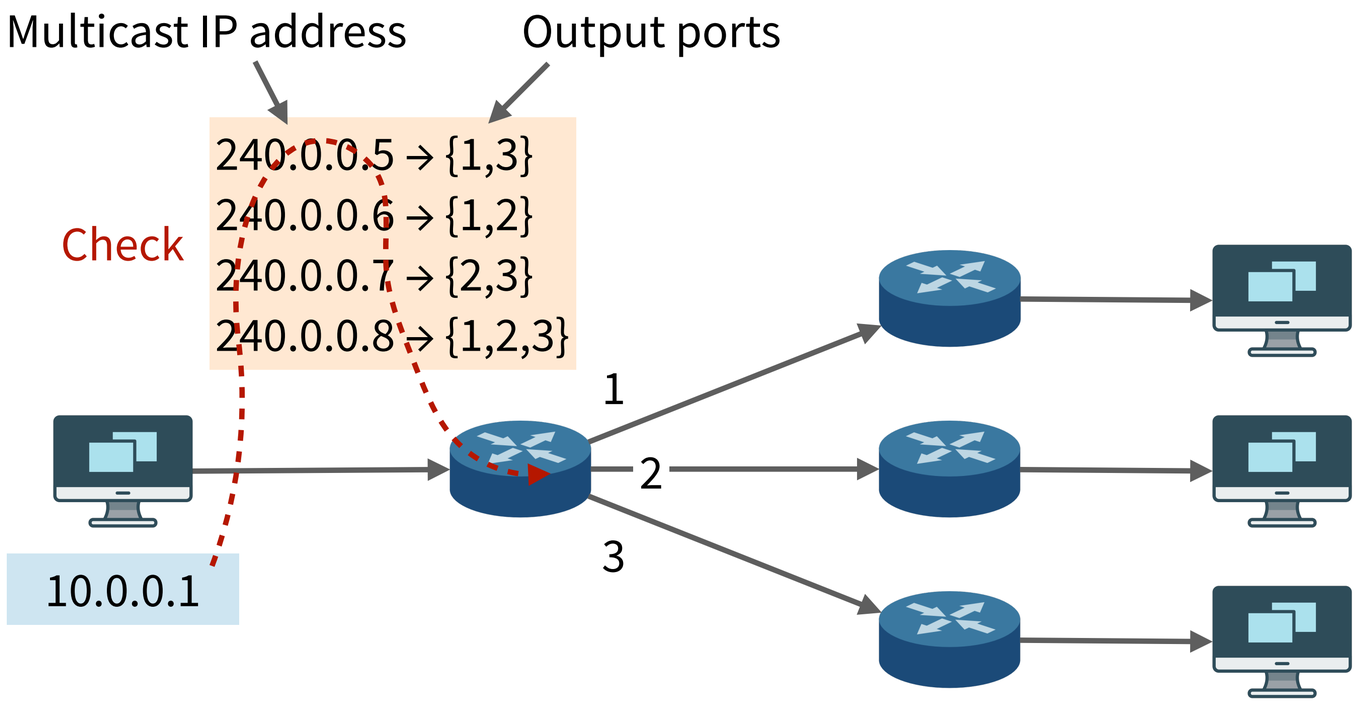
Example: IP multicast



You can implement these multicast groups with forwarding entries, but you may have a huge number of multicast groups that **do not fit on the router forwarding table**.

How to efficiently implement these groups in software?

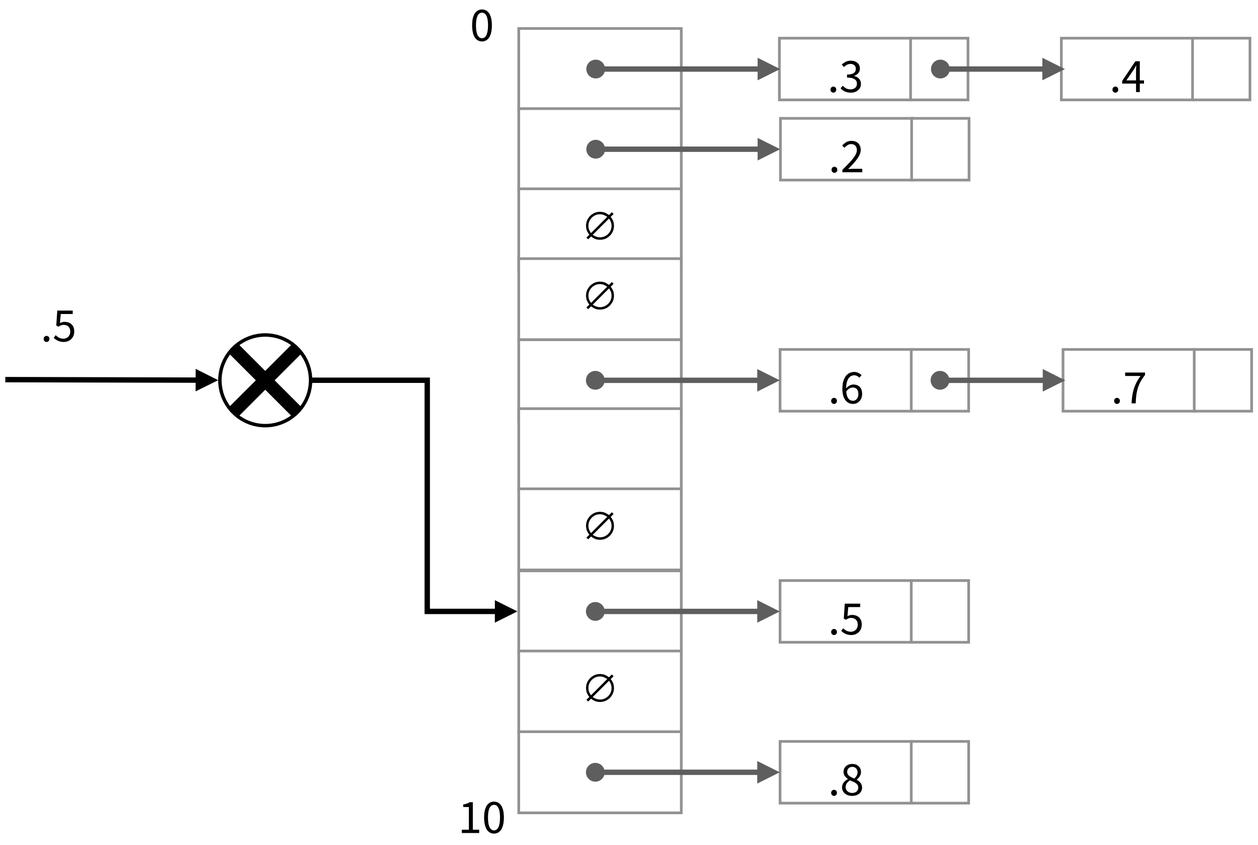
Example: IP multicast



Strategy #0 is to implement a list and check throughout the list upon each packet arrival. Assume the number of groups is N, the time complexity is O(N). The output is deterministic.

Improvement: separate chaining

Strategy #1



	List size
Average	N/M
Worst case	**N**

N: number of elements

M: number of cells

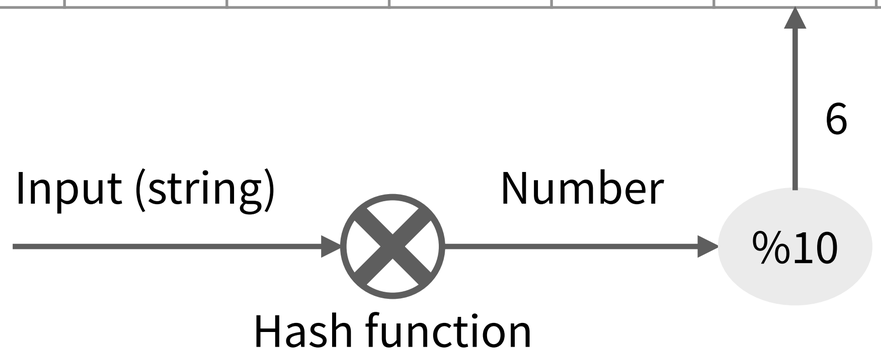
Pro: accurate and fast in the average case

Con: only works in hardware if there is a low number of elements

Bloom filter (simplest)

Strategy #2

Index	1	2	3	4	5	6	7	8	9	10
Cell						1				

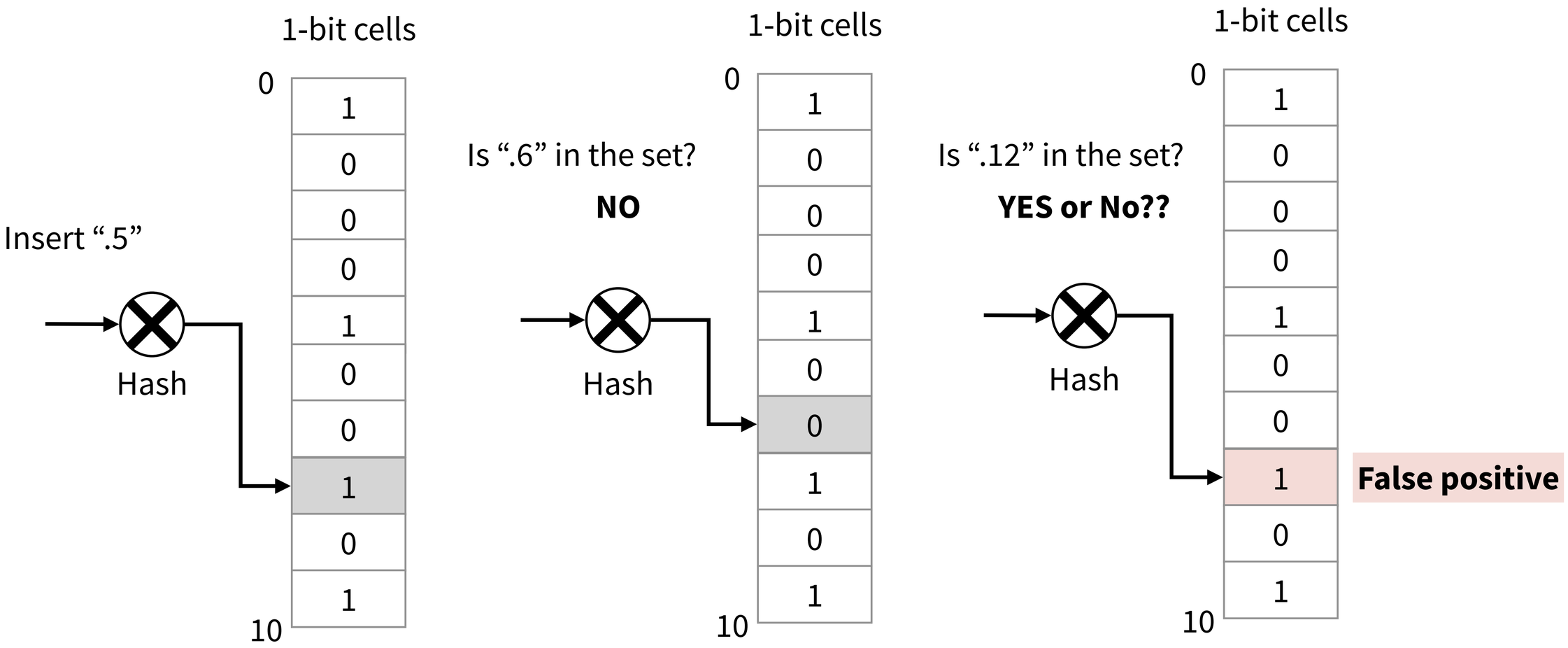


Check: if the matching cell is low, the input is not present for sure. However, if the cell is high, the input is probably present.

Insert: first apply a hash function and then apply modulo to map to the index space

Only supports two types of operations: “insert” and “check”, but sufficient for IP multicast group maintenance.

Bloom filter (simplest)



False positive

Assume we have N elements and M cells

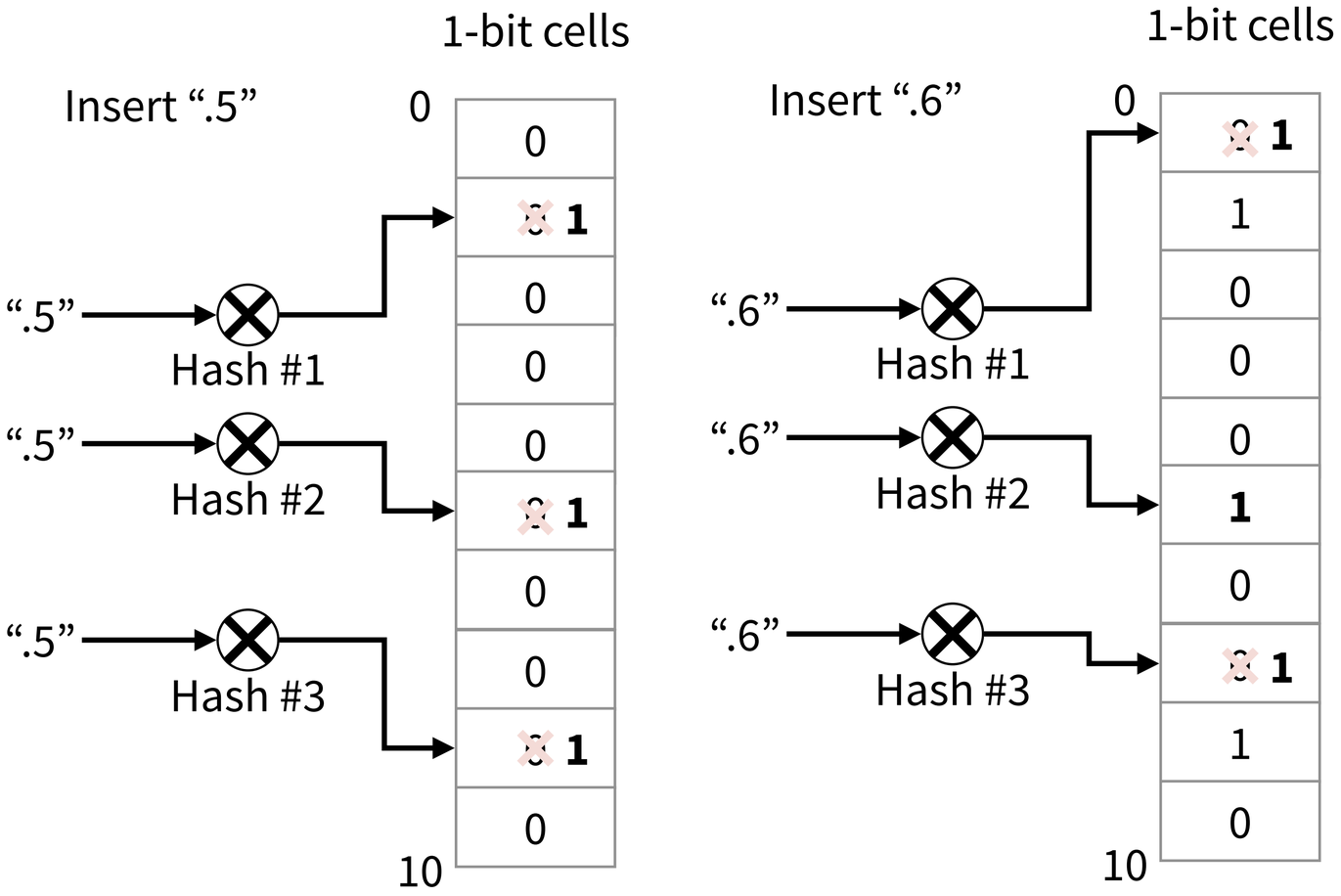
- Probability of an element to be mapped into a particular cell: 1/M
- Probability of an element not to be mapped into a particular cell: 1-1/M
- Probability of a cell to be 0: $(1-1/M)^N$
- **False positive rate (FPR):** $1-(1-1/M)^N$
- False negative rate?

# of elements	# of cells	FPR
1000	10,000	9.5%
1000	100,000	1%

Pro: simple and only one operation per insertion or query

Con: roughly 100x cells are required than the number of elements for 1% FPR

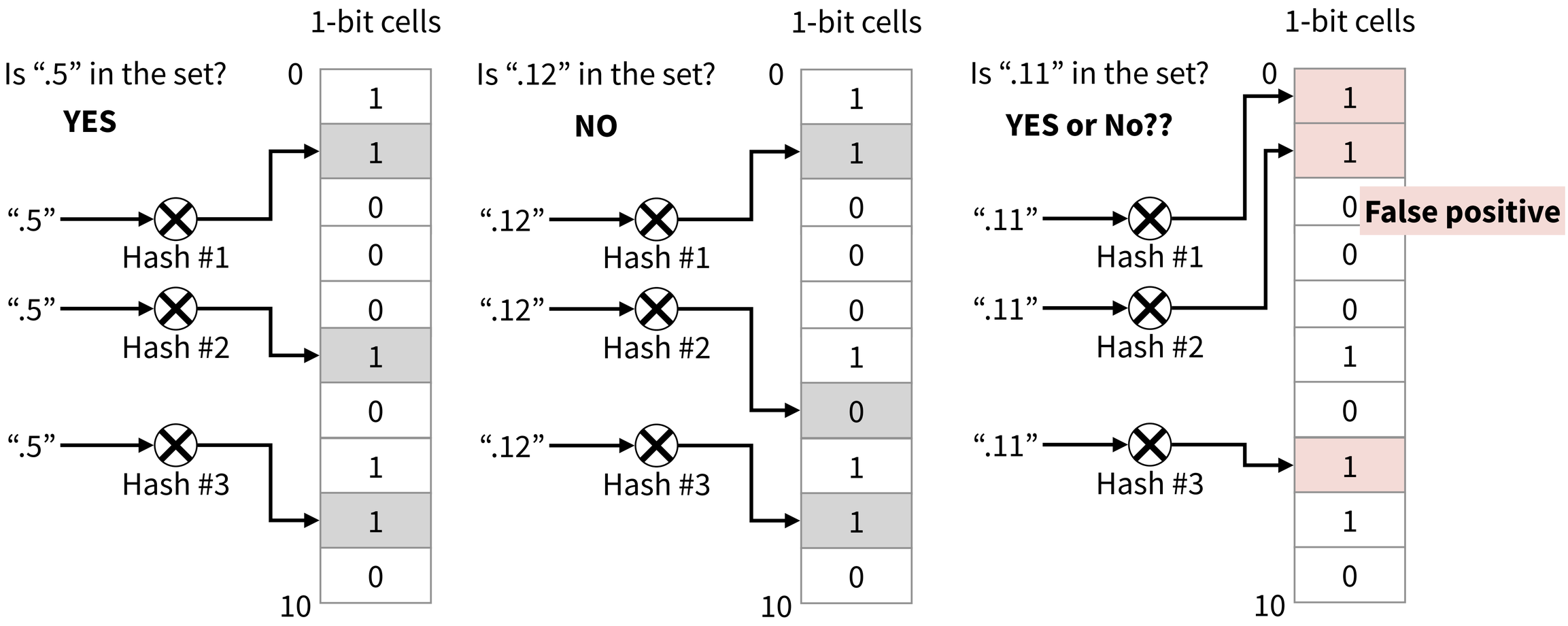
Bloom filter (general)



An element is considered in the set if **all** the hash values map to a cell with 1

An element is not in the set if **at least** one hash value maps to a cell with 0

False positive



False positive

Assume we have N elements, M cells and K hash functions

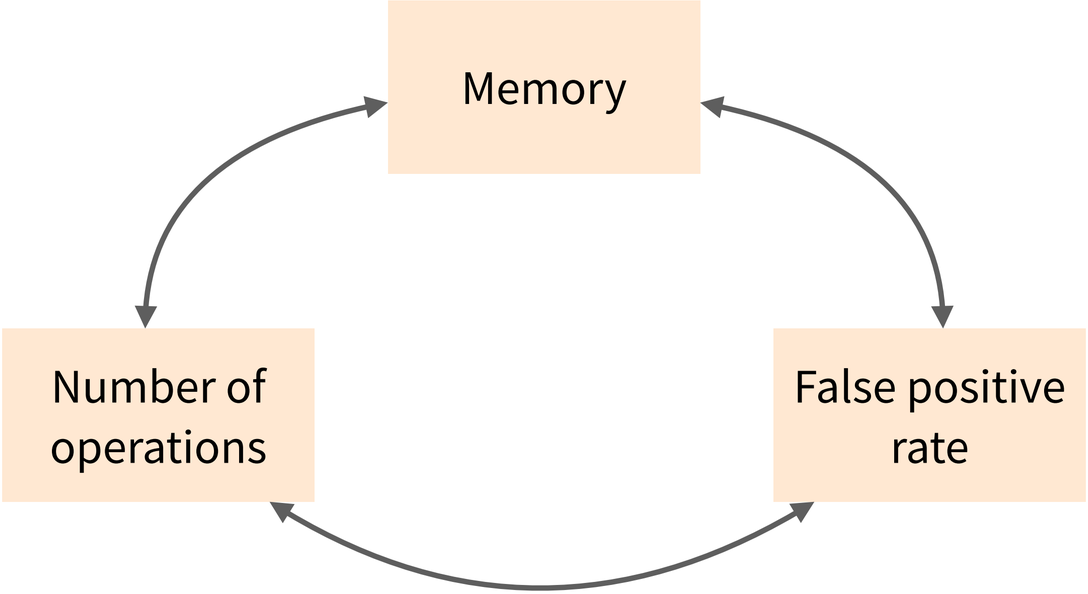
- Probability that one hash function returns the index of a particular cell: 1/M
- Probability that one hash function does not return the index of a particular cell: 1-1/M
- Probability of a cell to be 0 after K hashes over N elements: $(1-1/M)^{KN}$
- False positive rate: $(1-(1-1/M)^{KN})^{K}$
- False negative rate?

# of elements	# of cells	# of hash func	FPR
1000	10,000	7	0.82%
1000	100,000	7	≈0%

Pro: consumes roughly 10x less memory than the simple approach

Con: requires slightly more operations than the simple approach (7 hashes than just 1)

Tradeoff space



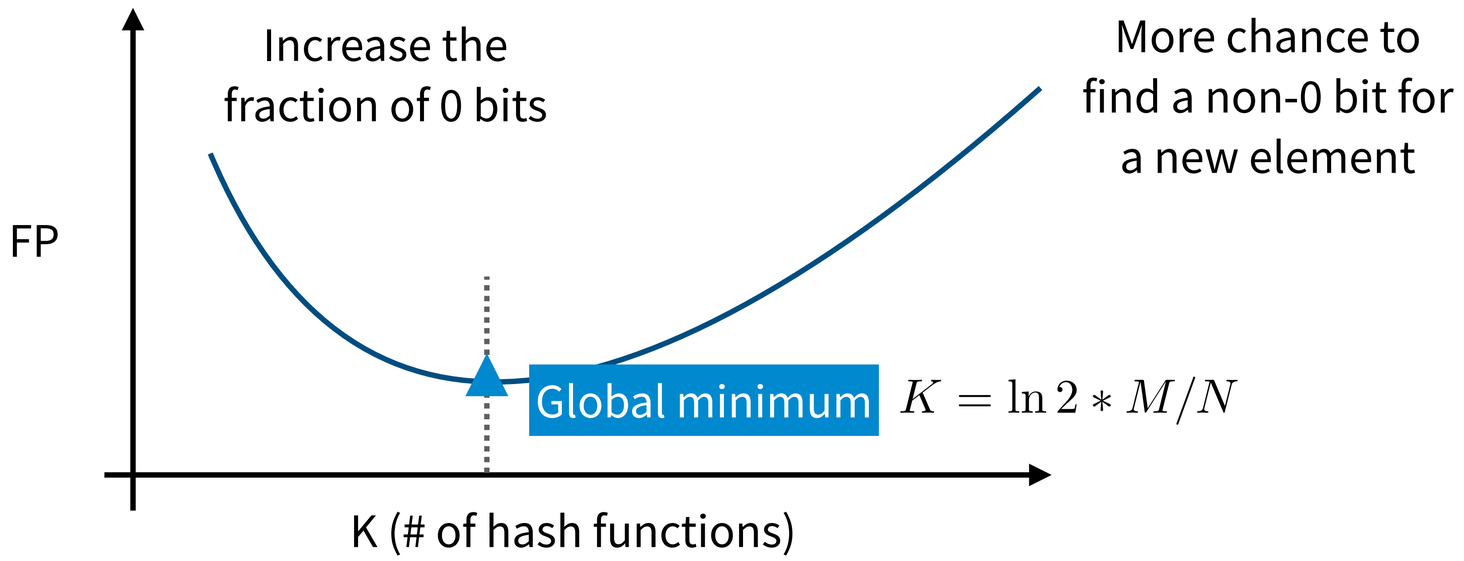
How to dimension your Bloom filter, e.g., deciding the number of cells and number of hash functions to satisfy a given false positive rate?

Dimensioning Bloom filters

N elements, **M** cells, **K** hash functions, and **FP** false positive rate

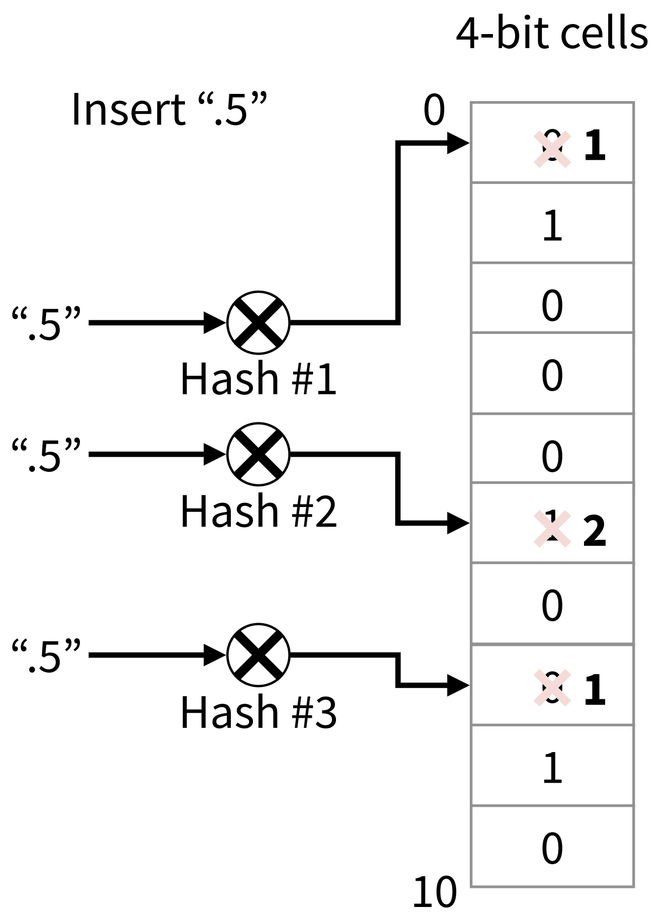
Asymptotic approximation

$$FP = (1 - (1 - \frac{1}{M})^{KN})^K \approx (1 - e^{-KN/M})^K$$

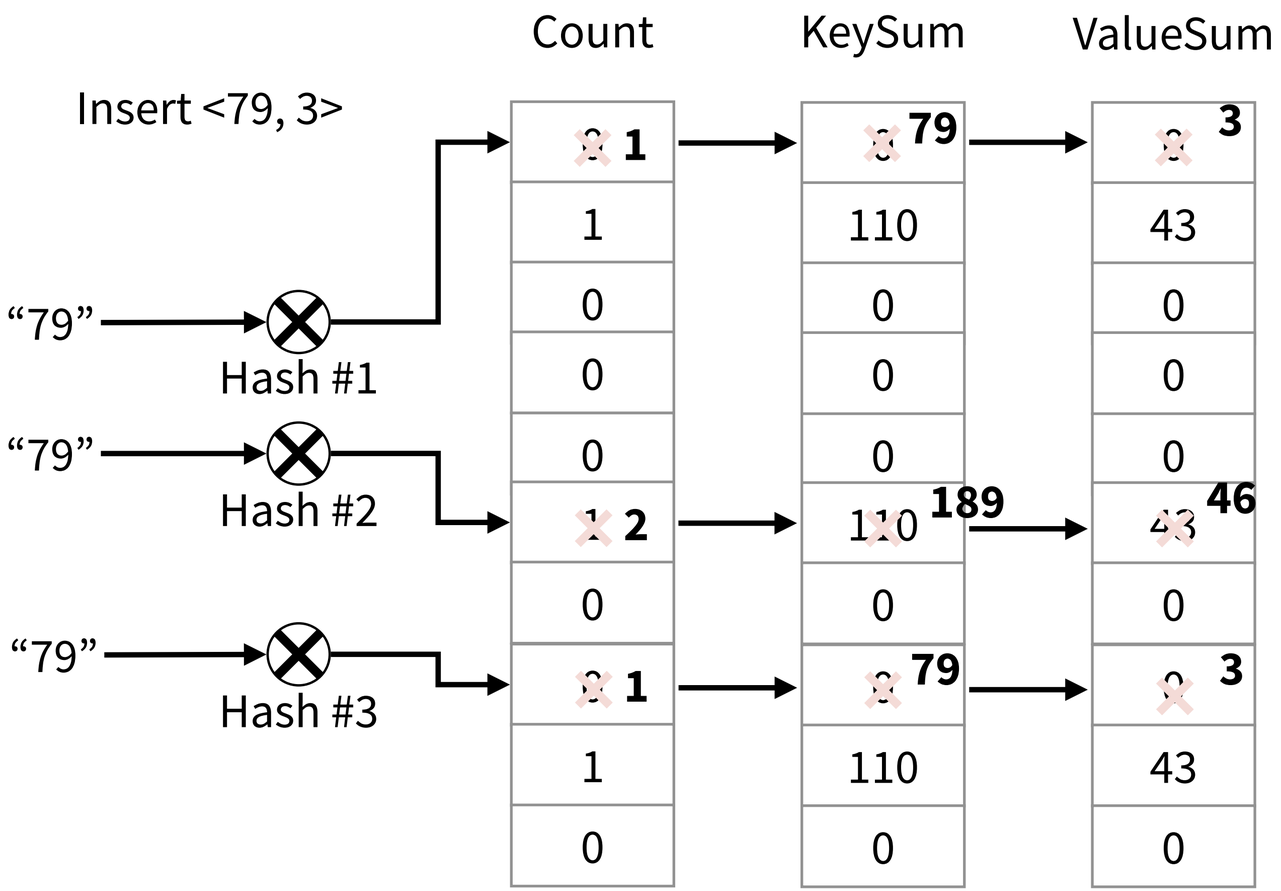


Bloom filter extensions

Counting Bloom filter



Invertible Bloom loopup tables (IBLT)



Limitations of Bloom filters

A Bloom filter is a streaming algorithm **answering specific questions** approximately

- Is X in the stream?
- What is in the stream?

What about other questions?

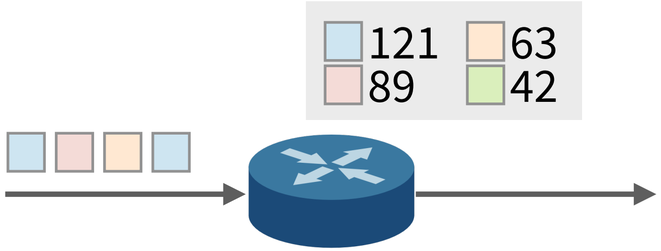
- How frequently does an element appear? (Count sketch, **CountMin sketch**)
- How many distinct elements are in the stream? (HyperLogLog sketch)
- What are the most frequent elements? (Count/CountMin + heap…)
- How many elements belong to a certain subnet? (SketchLearn)

In addition to packet flows, these questions apply to other domains like search engines and databases

Example: heavy hitter detection

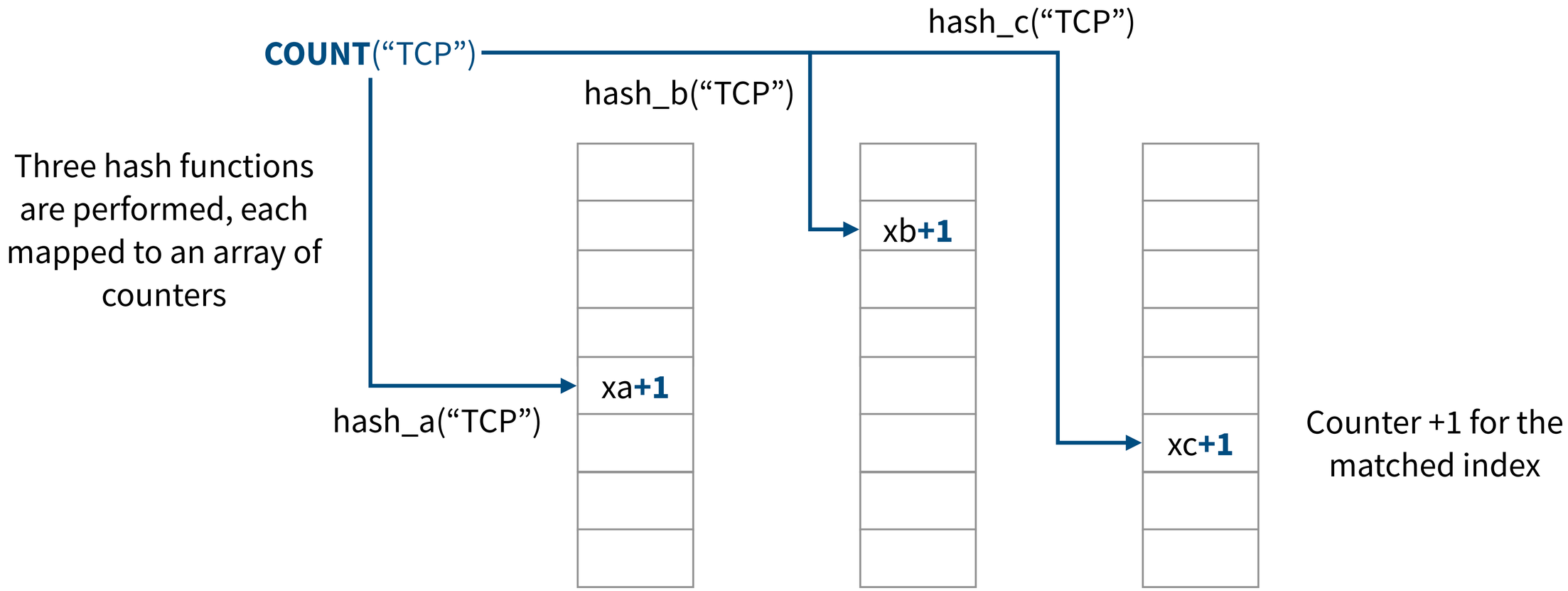
"Can you tell me what are the top 10 flows (in terms of traffic volume, #packets) that have passed through a given router?"

- A flow is defined by a 5-tuple: <source IP, destination IP, source port, destination port, protocol>
- Identifying the heavy hitters can help us understand the network performance better
- There might be thousands or even more flows on a single switch
- Switches are resource limited and you cannot create a separate counter for each individual flow

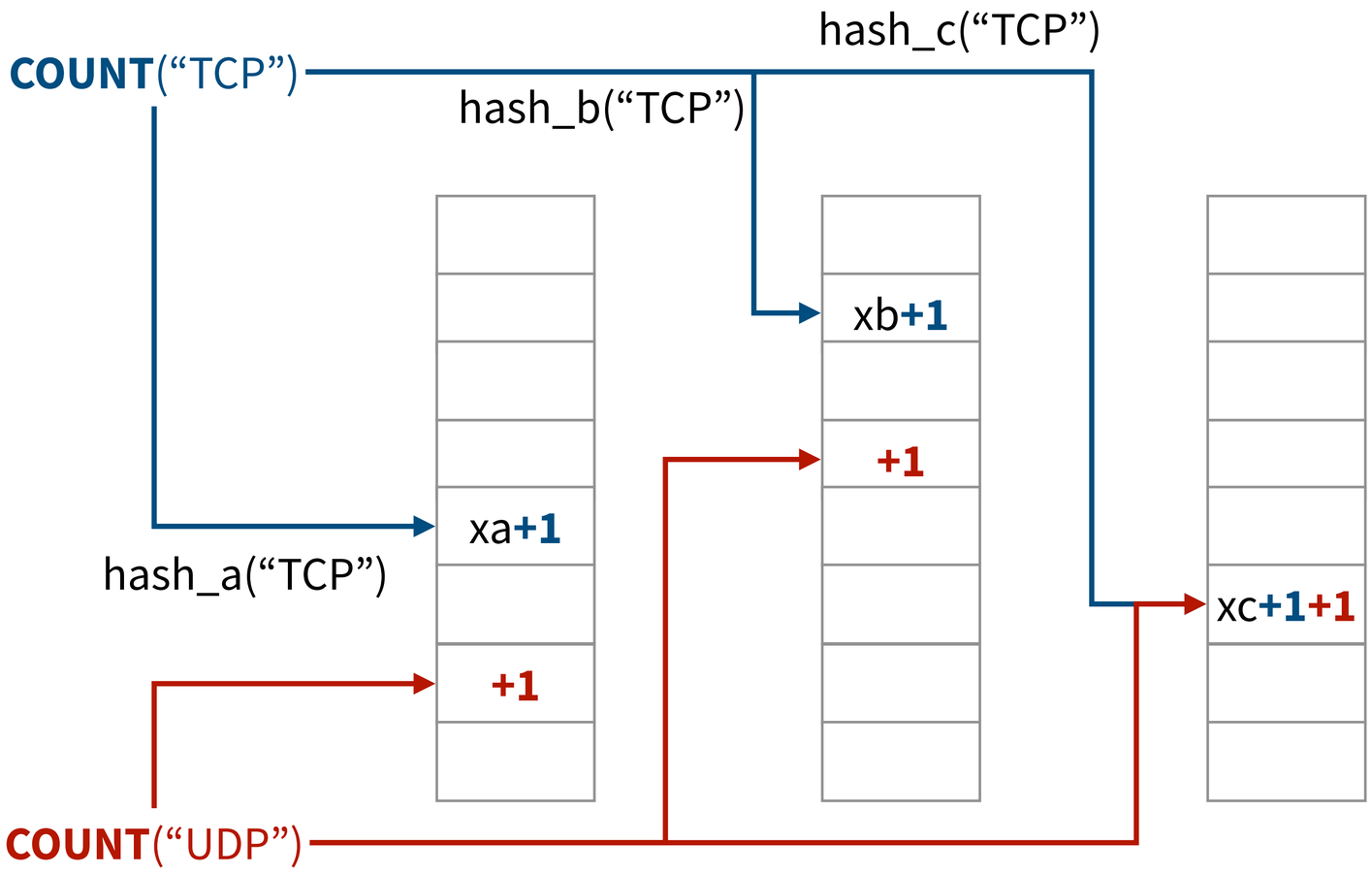


What is your idea about this problem?

CountMin sketch: providing frequency of elements

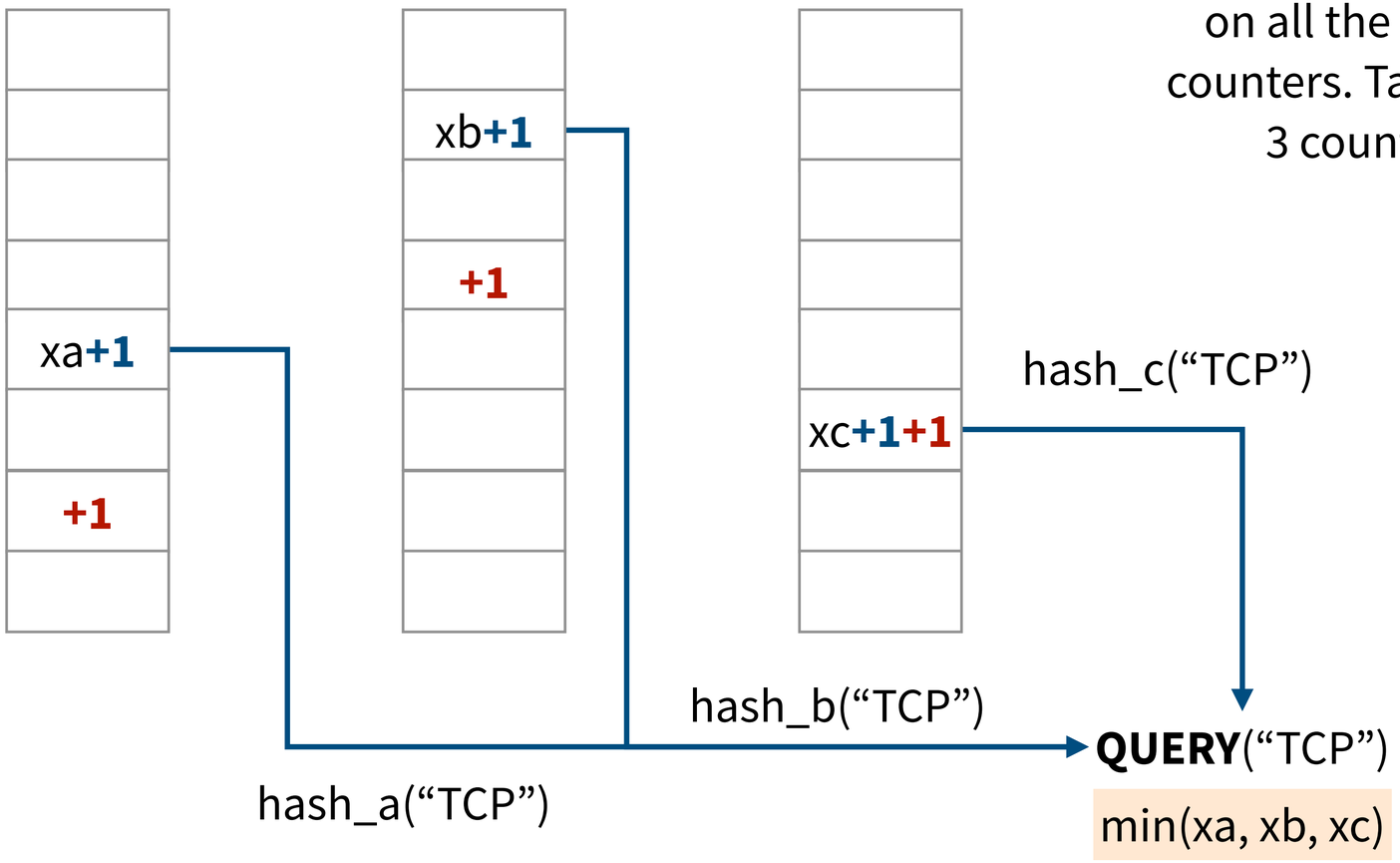


CountMin sketch: providing frequency of elements



CountMin sketch: providing frequency of elements

Perform the same hash functions on all the arrays and obtain the counters. Take the minimum of the 3 counters as the output.



Exercise

CountMin sketch

13	24	39	7	26
55	3	42	90	43
71	23	14	58	19

What would be the outcome of the CountMin sketch? Is that the lower-bound or the upper-bound of the real count?

CountMin sketch: performance

A CountMin sketch uses the same principles as a counting Bloom filter, but is designed to have **provable L1 error bounds** for frequency queries

$$Pr[\hat{x}_i - x_i \geq \varepsilon ||x||_1] \leq \delta$$

Estimated frequency

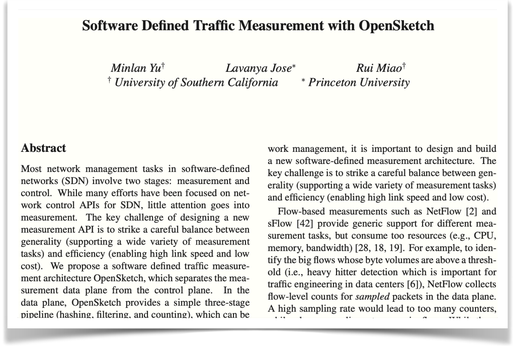
True frequency

Sum of frequencies

Favoring high-frequency elements: error is lower for high-frequency elements, but much higher for low-frequency elements

http://theory.stanford.edu/~tim/s15/l/l2.pdf

Sketches are the new dark



Software Defined Traffic Measurement with OpenSketch

Minlan Yu[†] Lavanya Jose[*] Rui Miao[†]
[†] University of Southern California [*] Princeton University

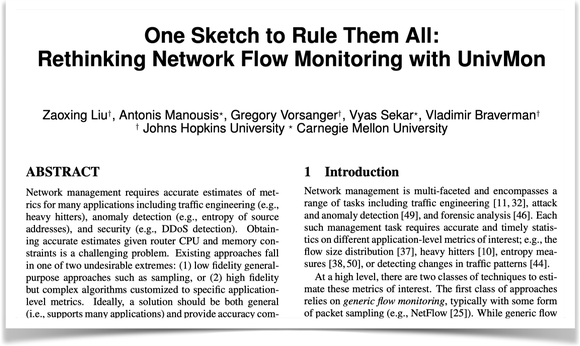
Abstract

Most network management tasks in software-defined networks (SDN) involve two stages: measurement and control. While many efforts have been focused on network control APIs for SDN, little attention goes into measurement. The key challenge of designing a new measurement API is to strike a careful balance between generality (supporting a wide variety of measurement tasks) and efficiency (enabling high link speed and low cost). We propose a software defined traffic measurement architecture OpenSketch, which separates the measurement data plane from the control plane. In the data plane, OpenSketch provides a simple three-stage pipeline (hashing, filtering, and counting), which can be

work management, it is important to design and build a new software-defined measurement architecture. The key challenge is to strike a careful balance between generality (supporting a wide variety of measurement tasks) and efficiency (enabling high link speed and low cost).

Flow-based measurements such as NetFlow [2] and sFlow [42] provide generic support for different measurement tasks, but consume too resources (e.g., CPU, memory, bandwidth) [28, 18, 19]. For example, to identify the big flows whose byte volumes are above a threshold (i.e., heavy hitter detection which is important for traffic engineering in data centers [6]), NetFlow collects flow-level counts for *sampled* packets in the data plane. A high sampling rate would lead to too many counters,

OpenSketch (NSDI'13)



One Sketch to Rule Them All:
Rethinking Network Flow Monitoring with UnivMon

Zaoxing Liu[†], Antonis Manousis[*], Gregory Vorsanger[†], Vyas Sekar[*], Vladimir Braverman[†]
[†] Johns Hopkins University [*] Carnegie Mellon University

ABSTRACT

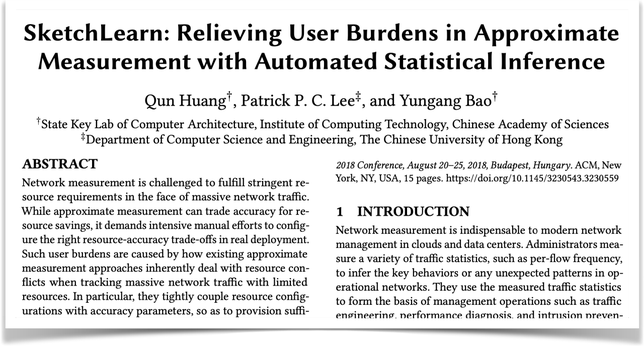
Network management requires accurate estimates of metrics for many applications including traffic engineering (e.g., heavy hitters), anomaly detection (e.g., entropy of source addresses), and security (e.g., DDoS detection). Obtaining accurate estimates given router CPU and memory constraints is a challenging problem. Existing approaches fall in one of two undesirable extremes: (1) low fidelity general-purpose approaches such as sampling, or (2) high fidelity but complex algorithms customized to specific application-level metrics. Ideally, a solution should be both general (i.e., supports many applications) and provide accuracy com-

1 Introduction

Network management is multi-faceted and encompasses a range of tasks including traffic engineering [11, 32], attack and anomaly detection [49], and forensic analysis [46]. Each such management task requires accurate and timely statistics on different application-level metrics of interest; e.g., the flow size distribution [37], heavy hitters [10], entropy measures [38, 50], or detecting changes in traffic patterns [44].

At a high level, there are two classes of techniques to estimate these metrics of interest. The first class of approaches relies on *generic flow monitoring*, typically with some form of packet sampling (e.g., NetFlow [25]). While generic flow

UnivMon (SIGCOMM'16)



SketchLearn: Relieving User Burdens in Approximate Measurement with Automated Statistical Inference

Qun Huang[†], Patrick P. C. Lee[‡], and Yungang Bao[†]
[†]State Key Lab of Computer Architecture, Institute of Computing Technology, Chinese Academy of Sciences
[‡]Department of Computer Science and Engineering, The Chinese University of Hong Kong

ABSTRACT

Network measurement is challenged to fulfill stringent resource requirements in the face of massive network traffic. While approximate measurement can trade accuracy for resource savings, it demands intensive manual efforts to configure the right resource-accuracy trade-offs in real deployment. Such user burdens are caused by how existing approximate measurement approaches inherently deal with resource conflicts when tracking massive network traffic with limited resources. In particular, they tightly couple resource configurations with accuracy parameters, so as to provision suffi-

2018 Conference, August 20–25, 2018, Budapest, Hungary. ACM, New York, NY, USA, 15 pages. https://doi.org/10.1145/3230543.3230559

1 INTRODUCTION

Network measurement is indispensable to modern network management in clouds and data centers. Administrators measure a variety of traffic statistics, such as per-flow frequency, to infer the key behaviors or any unexpected patterns in operational networks. They use the measured traffic statistics to form the basis of management operations such as traffic engineering, performance diagnosis, and intrusion preven-

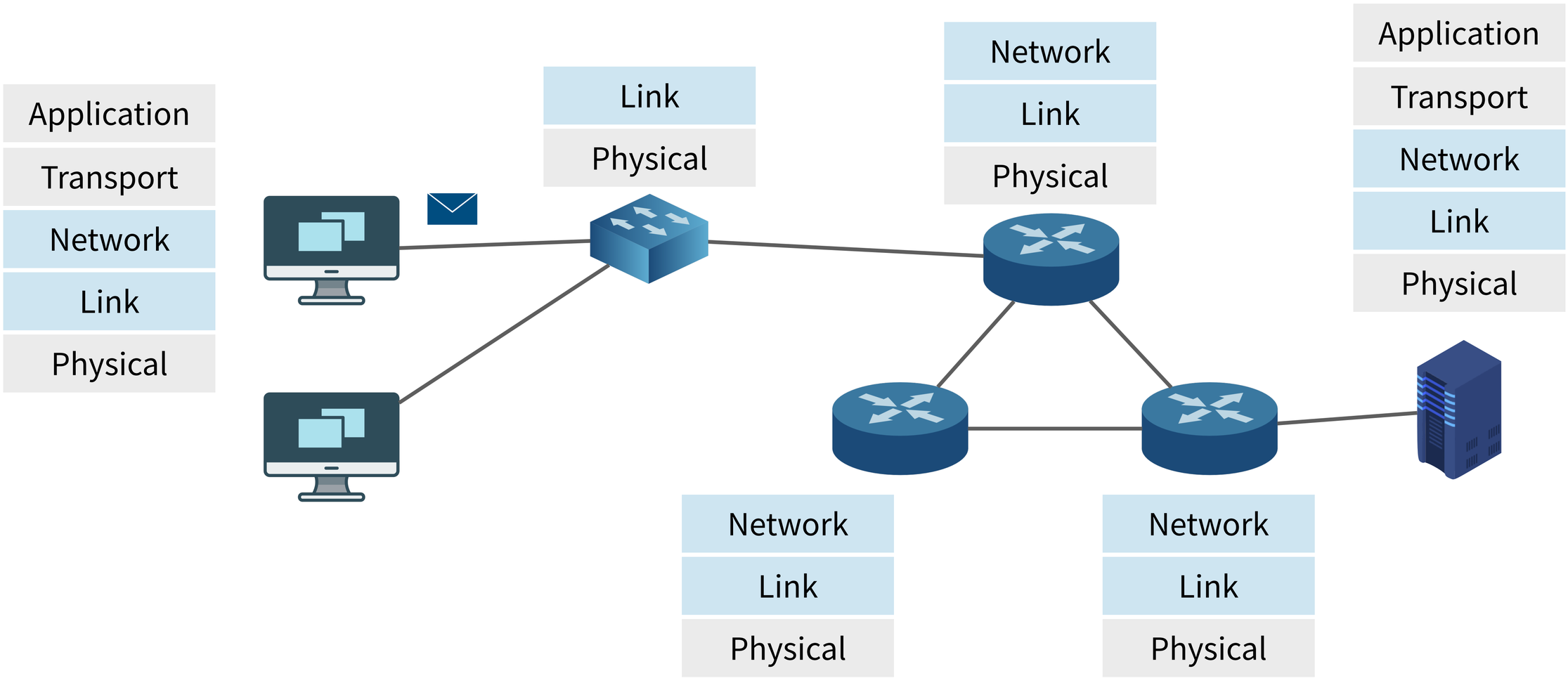
SketchLearn (SIGCOMM'18)

Summary

What do we want from the network and how to achieve them? (principles)

- Performance, fairness, reliability, flexibility, etc.
- Principles: the **end-to-end argument**, the **fate-sharing** principle, **packet vs. circuit switching**

Basic data structures for networking

- Networking tasks: traffic monitoring, classification, membership determination
- **Bloom filter:** answering if an element appears in a set or what are in a set
- **CountMin sketch:** answering how many times an element appears in a set

Next time: the journey of a packet